EECS 483: Compiler Construction

Lecture 17:
Register Allocation Part 2: Graph Coloring and Code Generation

March 19
Winter Semester 2025

Announcements

- Assignment 4 is released. Due April 3rd

  Covers heap-allocated objects (arrays) and dynamic typing

Register Allocation

3 Steps

1. **Liveness analysis**: identify when each variable's value is needed in the program
2. **Conflict analysis**: identify which variables interfere with each other
3. **Graph Coloring**: assign variables to registers so that interfering registers are assigned different registers.
    1. Spilling: if necessary, assign some variables to stack slots

Liveness Analysis: Example

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
B     a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

In the sub-expression B, which variables are

In scope:

Syntactically occurring:

Live:

Liveness Analysis: Example

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
B     a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

In the sub-expression B, which variables are

In scope: `x,y,z,i,a,i'`

Syntactically occurring: `x,a,a',i'`

Live: `x,z,a,i'`

Liveness Analysis: Example

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
B     a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

In the sub-expression B, which variables are

In scope: `x,y,z,i,a,i'`

Syntactically occurring: `x,a,a',i'`

Live: `x,z,a,i'`

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 0	Round 1
1:	{ }	?
2:	{ }	?
3:	{ }	?
4:	{ }	?
5/6:	{ }	?
7:	{ }	?
8/9:	{ }	?
10:	{ }	?
11:	{ }	?

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 0	Round 1
1:	{ }	{x,z}
2:	{ }	{y}
3:	{ }	{a,i,x,z}
4:	{ }	{a,b,i,x,z}
5/6:	{ }	{a,z}
7:	{ }	{r}
8/9:	{ }	{a,i,x}
10:	{ }	{a,i',x}
11:	{ }	{a',i'}

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 1	Round 2
1:	{x,z}	?
2:	{y}	?
3:	{a,i,x,z}	?
4:	{a,b,i,x,z}	?
5/6:	{a,z}	?
7:	{r}	?
8/9:	{a,i,x}	?
10:	{a,i',x}	?
11:	{a',i'}	?

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 1	Round 2
1:	{x,z}	{x,z}
2:	{y}	{x,y,z}
3:	{a,i,x,z}	{a,i,x,z}
4:	{a,b,i,x,z}	{a,b,i,x,z}
5/6:	{a,z}	{a,z}
7:	{r}	{r}
8/9:	{a,i,x}	{a,i,x,z}
10:	{a,i',x}	{a,i',x,z}
11:	{a',i'}	{a',i',x,z}

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 2	Round 3
1:	{x,z}	1: ?
2:	{x,y,z}	2: ?
3:	{a,i,x,z}	3: ?
4:	{a,b,i,x,z}	4: ?
5/6:	{a,z}	5/6: ?
7:	{r}	7: ?
8/9:	{a,i,x,z}	8/9: ?
10:	{a,i',x,z}	10: ?
11:	{a',i',x,z}	11: ?

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

	Round 2	Round 3
1:	{x,z}	{x,z}
2:	{x,y,z}	{x,y,z}
3:	{a,i,x,z}	{a,i,x,z}
4:	{a,b,i,x,z}	{a,b,i,x,z}
5/6:	{a,z}	{a,z}
7:	{r}	{r}
8/9:	{a,i,x,z}	{a,i,x,z}
10:	{a,i',x,z}	{a,i',x,z}
11:	{a',i',x,z}	{a',i',x,z}

Liveness Analysis: Example

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
 B   10 a' = a + x
     11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

1:	{x,z}
2:	{x,y,z}
3:	{a,i,x,z}
4:	{a,b,i,x,z}
5/6:	{a,z}
7:	{r}
8/9:	{a,i,x,z}
10:	{a,i',x,z}
11:	{a',i',x,z}

In the sub-expression B, which variables are

In scope: x,y,z,i,a,i'

Syntactically occurring: x,a,a',i'

Live: x,**z**,a,i'

Implementation Concerns

How to store live sets?

- Add annotation metadata to the SSA AST
  - `init_liveness(e: BB<T>) -> BB<HashSet<String>>`
  - `update_liveness(e: BB<HashSet<String>>) -> BB<HashSet<String>>`
- iterate until you reach a fixed point
  - `update_liveness(b) == b`

Conflict Analysis

Once we know when we need the value of each variable, we determine which variables cannot be assigned the same register

2 variables truly conflict when

- They are live at the same time
- with different values

Err on the side of *too many* conflicts.

Conflict Analysis

Simple approach:

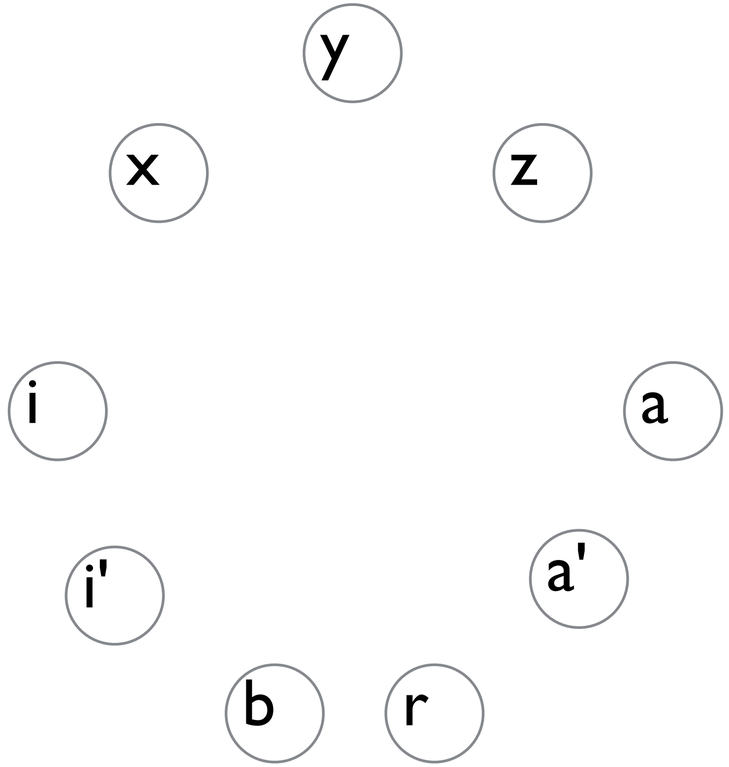
- Initialize the graph with all variables in the program
- Whenever a variable is assigned to, add conflicts with all the live variables
- Also, parameters of a block all must mutually conflict, as they are assigned to simultaneously.

This is an overapproximation to true conflicts

Conflict Analysis

```
f(x,y,z):
 1 loop(i,a):
   5 thn():
     6 r = a * z
     7 ret r
   8 els():
     9 i' = i - 1
    10 a' = a + x
    11 br loop(i', a')
   3 b = i == 0
   4 cbr b thn() els()
 2 br loop(y, 0)
```

1:	{x,z}
2:	{x,y,z}
3:	{a,i,x,z}
4:	{a,b,i,x,z}
5/6:	{a,z}
7:	{r}
8/9:	{a,i,x,z}
10:	{a,i',x,z}
11:	{a',i',x,z}



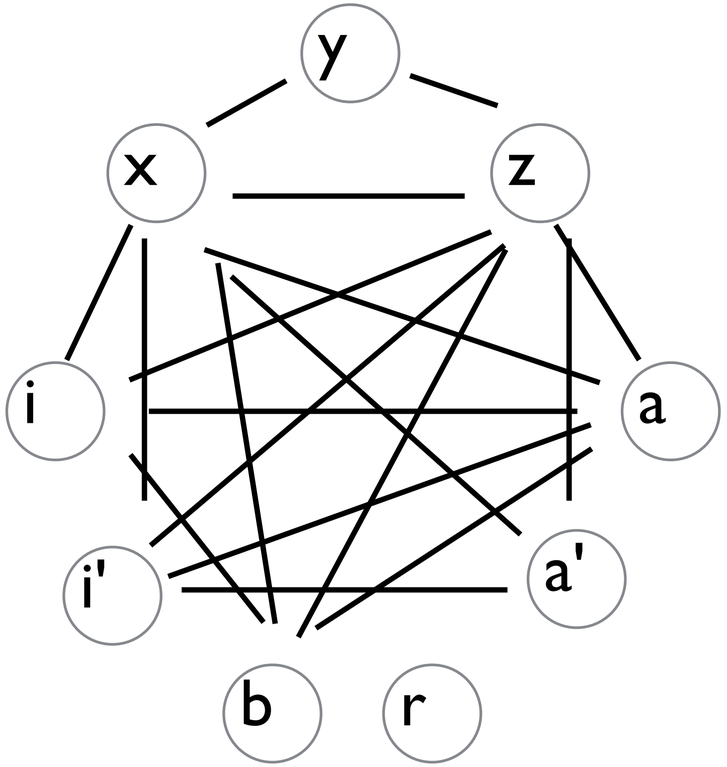
SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Liveness Info

```
1:    {x,z}
2:    {x,y,z}
3:    {a,i,x,z}
4:    {a,b,i,x,z}
5/6:  {a,z}
7:    {r}
8/9:  {a,i,x,z}
10:   {a,i',x,z}
11:   {a',i',x,z}
```

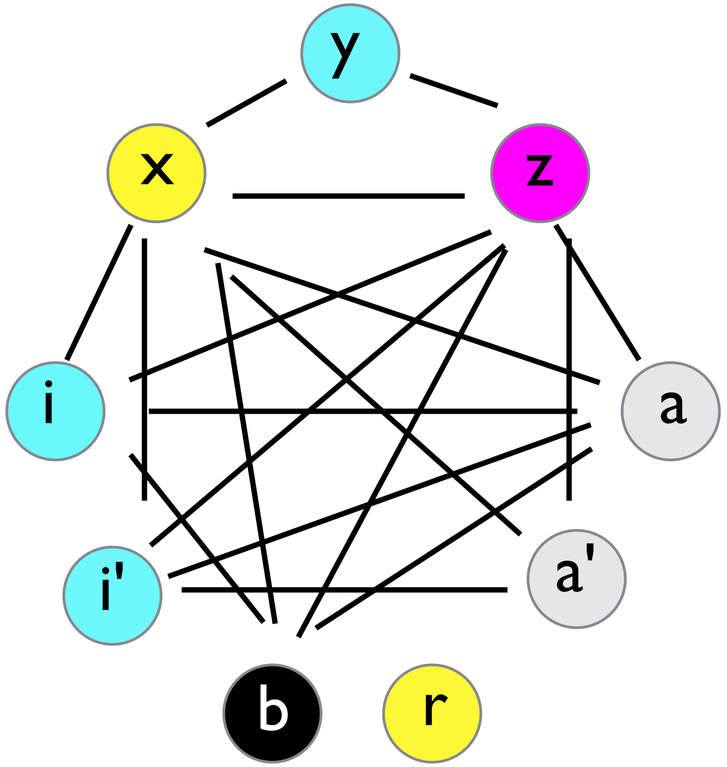
Interference Graph

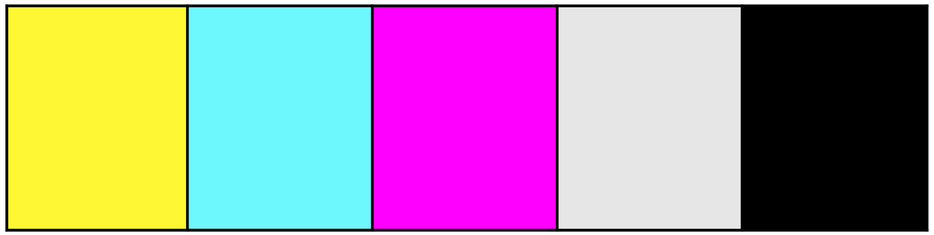


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

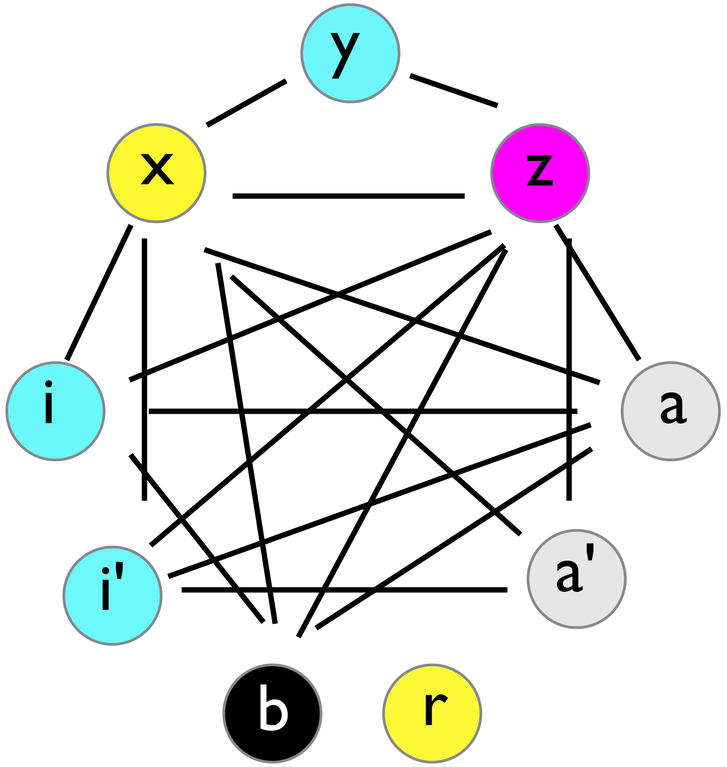




SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph



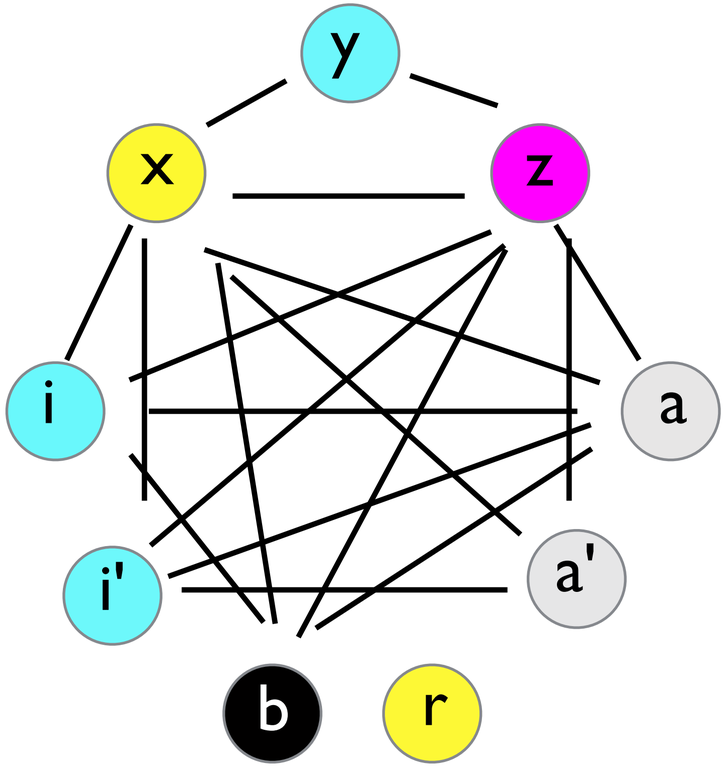
rax	rsi	rdi	rcx	rdx

SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

```
f:
  mov rcx, 0
  jmp loop
loop:
  cmp rsi, 0
  mov rdx, 0
  sete rdx
  cmp rdx 0
  jne thn
  jmp els
thn:
  mov rax, rcx
  imul rax, rdi
  ret
els:
  sub rsi, 1
  add rcx, rax
  jmp loop
```

Interference Graph



rax	rsi	rdi	rcx	rdx

Graph Coloring Register Allocation

Given our register interference graph, want to assign a register to each variable so that no interfering variables are assigned the same register.

Equivalent to graph coloring of the interference graph

- think of each register as a "color" and we want to paint each node so that no adjacent nodes are the same color.

Efficient algorithm for graph coloring -> efficient algorithm for register allocation!

Graph Coloring is Hard

Determining a whether a graph is k-colorable is NP-complete for k > 2.

- So no polytime algorithm is known

Does that mean register allocation is NP-hard?

Is Register Allocation Hard?

Chaitin et al, "Register allocation via coloring", *Computer Languages* 1981

- Showed that the register allocation problem for a language with assignments and arbitrary control flow (goto) is **equivalent** to graph coloring
- every graph arises as the interference graph of some program

So register allocation of an imperative language with goto is NP complete.

- But our programs are more restrictive: SSA form...

Chaitin's Algorithm

Chaitin's Algorithm

Chaitin's algorithm is efficient ($O(|V| + |E|)$), simple to implement

Chaitin's Algorithm

Simple recursive algorithm for constructing a valid coloring.

Fix an ordering on the nodes of the graph.

- Base case: if the graph is empty, it is trivially colorable
- Recursive case:
  - Remove a node from the graph, and all of its edges
  - Recursively color the remaining graph
  - Add the node and its edges back to the graph, pick a color that is different from all of its neighbors. If the neighbors have exhausted all of the colors, "spill"

Chaitin's Algorithm

Chaitin's algorithm is efficient (O(|V| + |E|), simple to implement

- How good the coloring is depends on the order we color the nodes to the graph
  - called the **elimination ordering**

Chaitin's Algorithm

Chaitin's algorithm is efficient (O(|V| + |E|), simple to implement

- How good the coloring is depends on the order we color the nodes to the graph
  - called the **elimination ordering**
- Theorem: For every graph, there is an elimination ordering such that Chaitin's algorithm produces an optimal coloring
  - therefore finding this optimal elimination ordering for a general graph is NP-complete

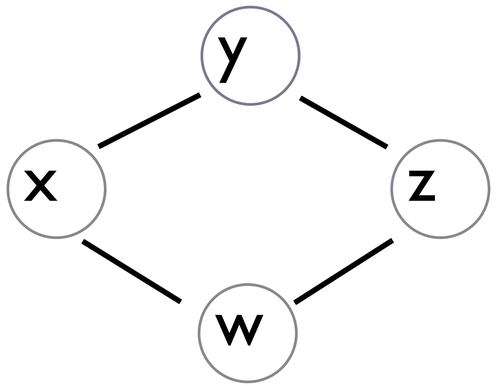
Graph Coloring SSA Programs

Hack et al, "Register Allocation for Programs in SSA-Form", *Compiler Construction* 2006

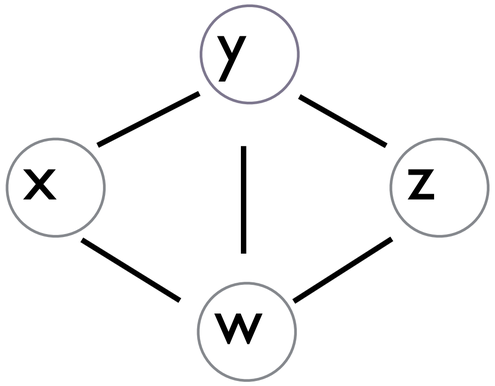
Graph Coloring SSA Programs

Hack et al, "Register Allocation for Programs in SSA-Form", *Compiler Construction* 2006

- The interference graphs of an SSA program are all **chordal**
  - Every cycle >= 4 nodes has a **chord**



Not chordal

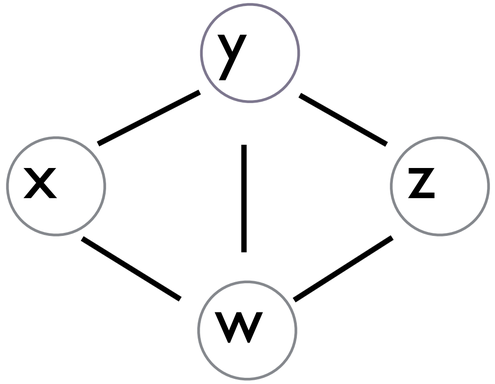


chordal

Coloring Chordal Graphs

Theorem: A graph is chordal iff it has a **perfect elimination ordering** (PEO)

- a total ordering of nodes v1,v2,v3,... such that for each vi, vi forms a clique with all its neighbors earlier in the order
- Chaitin's algo produces an optimal coloring if we use a PEO



x,y,z,w

not perfect: N(w) non-clique

w,x,y,z

perfect

Coloring Chordal Graphs

Theorem: A graph is chordal iff it has a **perfect elimination ordering (**PEO)

Theorem: A graph is chordal iff it is the intersection graph of a group of subtrees of a tree

- In an SSA program, each variable's liveness is a subtree of the AST
- The interference graph is exactly the intersection graph of those subtrees
- Therefore, the interference graph of an SSA program is chordal!

SSA Interference Graphs are Chordal!

Every SSA Interference Graph is chordal

Chaitin's algorithm computes optimal coloring given a PEO

So we can color SSA interference graphs if we can find a PEO

SSA programs have a perfect elimination ordering that is easy to compute:

"in-scope" or "dominance" relation

a variable x dominates y if x in scope when y is defined (includes simultaneous binding)

- x's definition is "closer to the root" of the AST than y
- easy to compute: pre-order traversal of the nodes

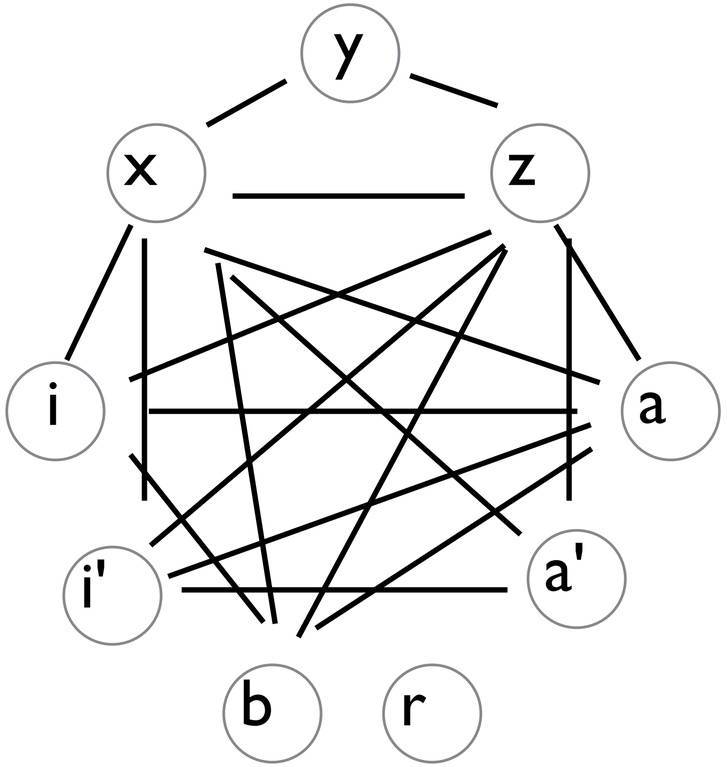
SSA Program

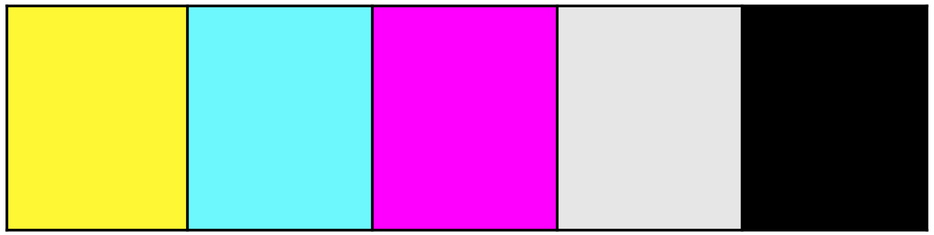
```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Pick a PEO:

variables should be colored after anything that was already in scope

Interference Graph

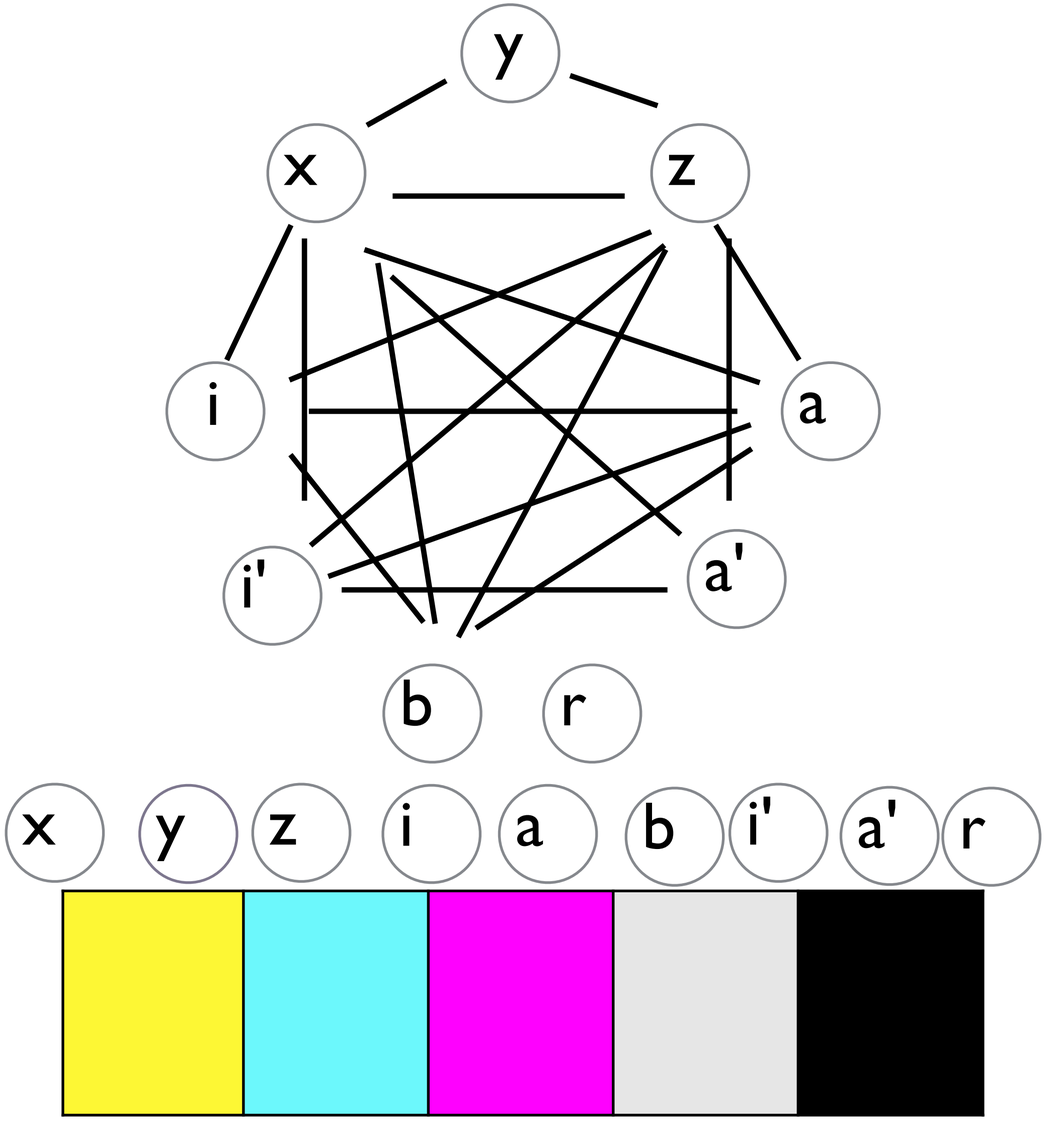




SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

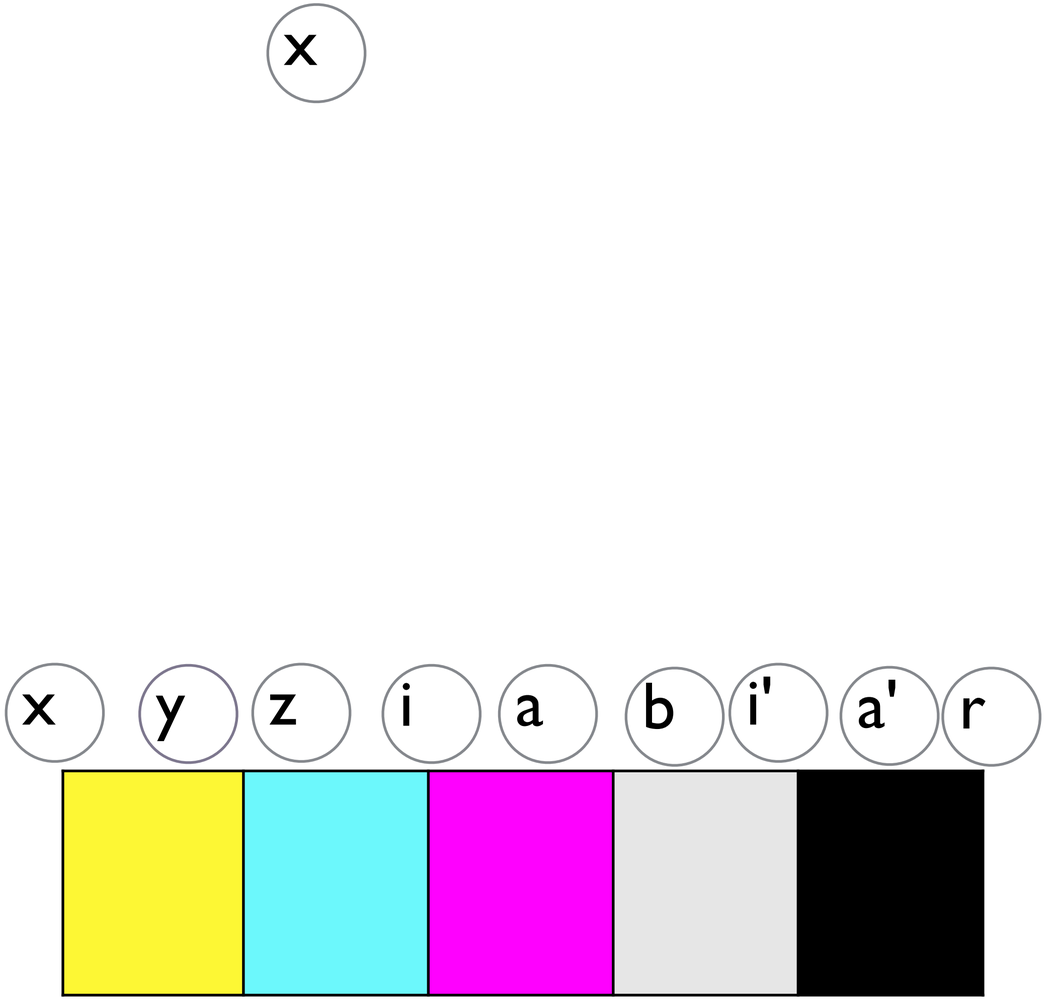
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

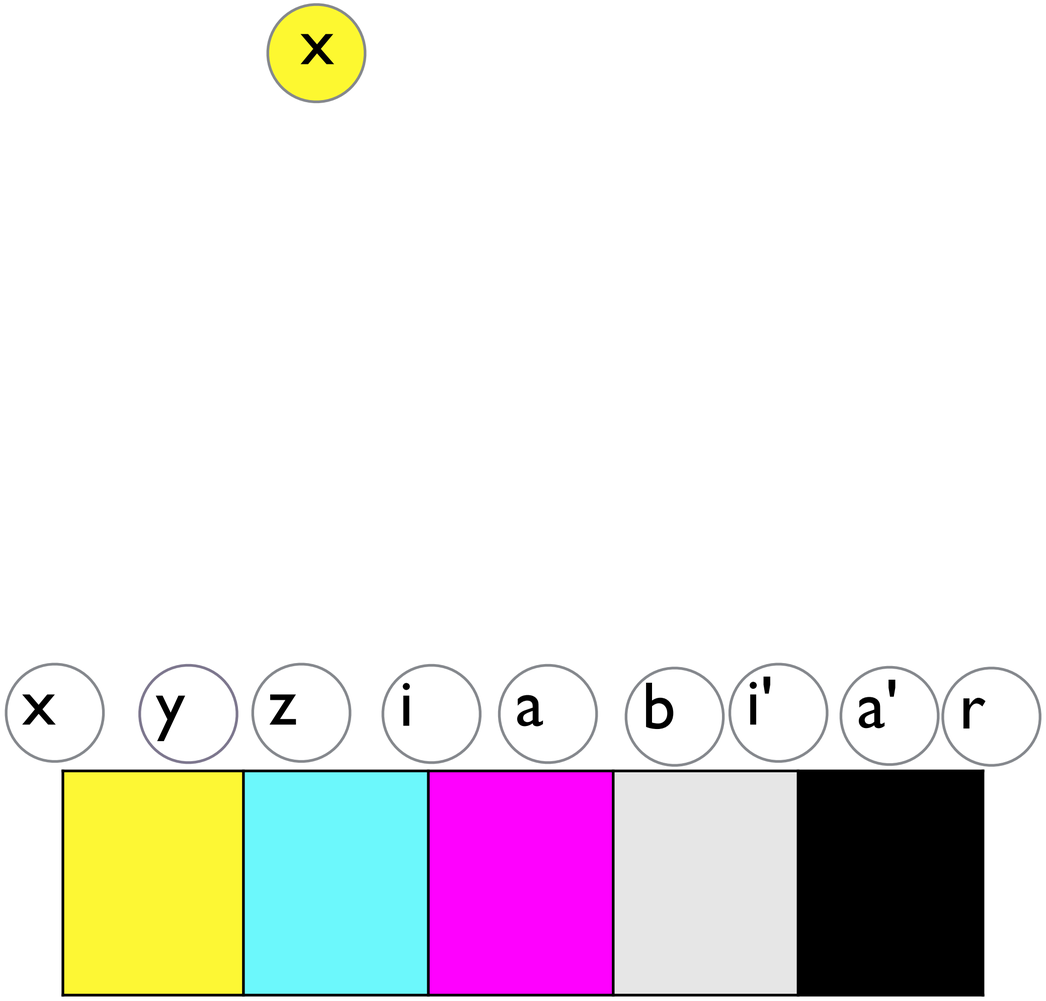
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

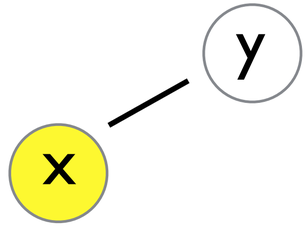
Interference Graph

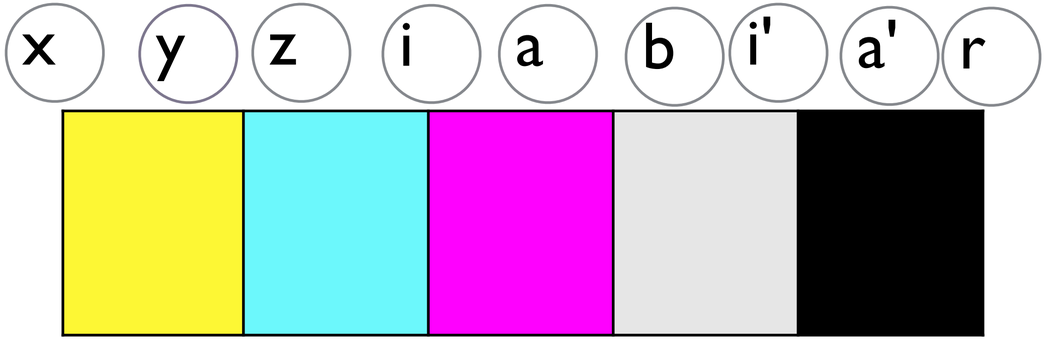


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

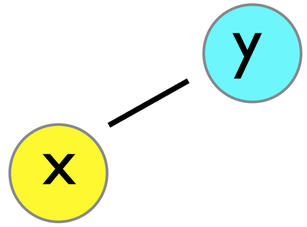


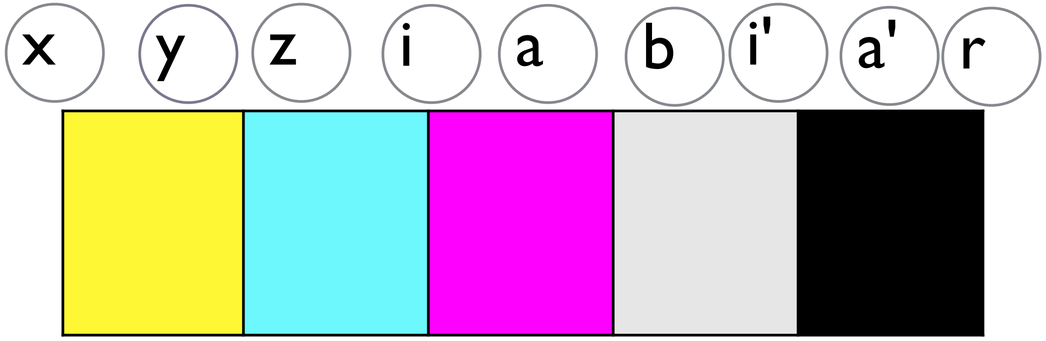


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

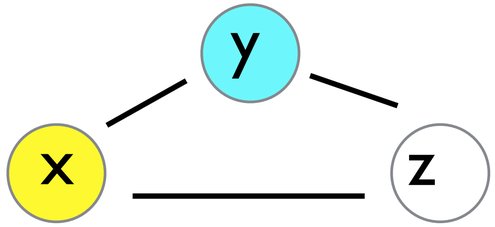


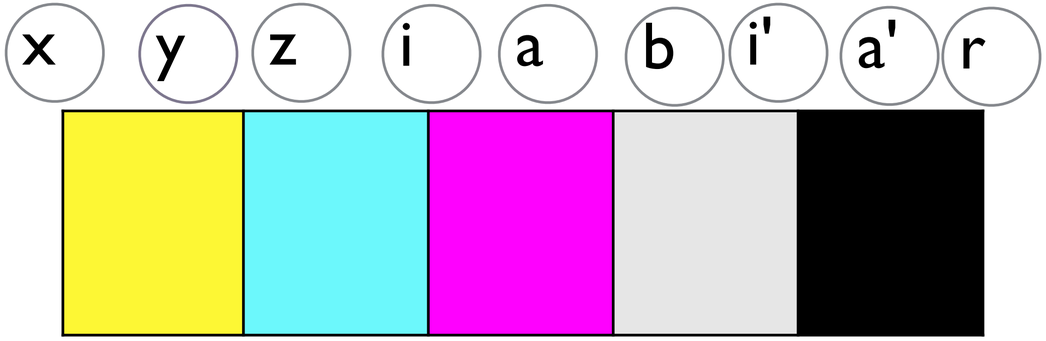


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

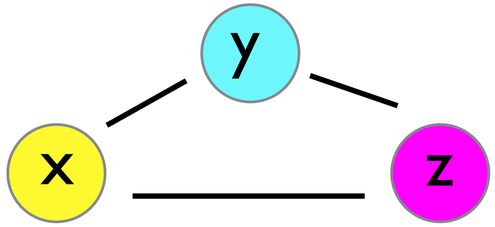


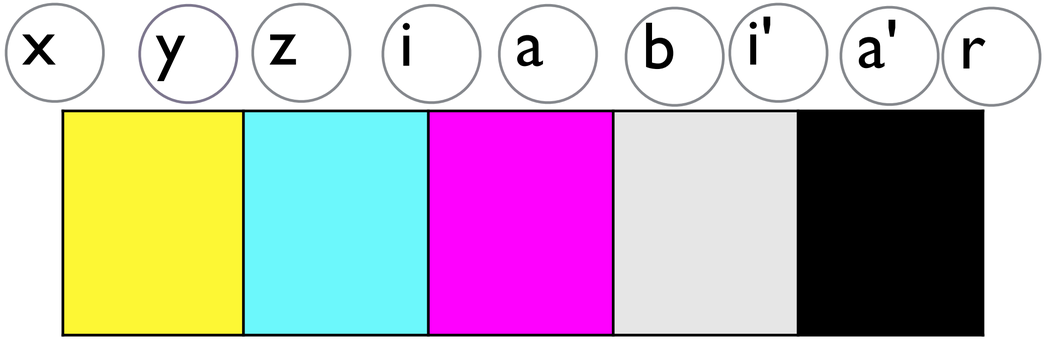


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

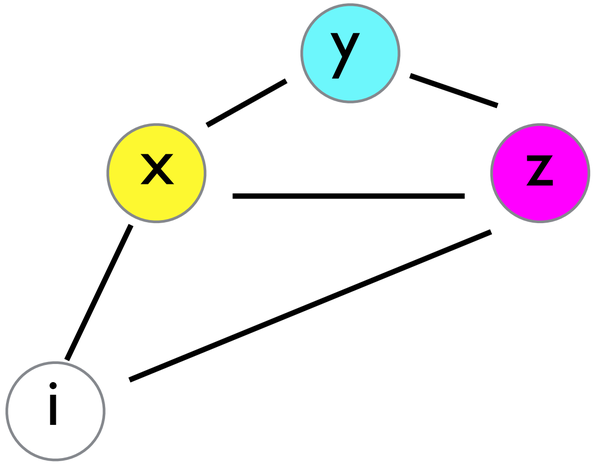


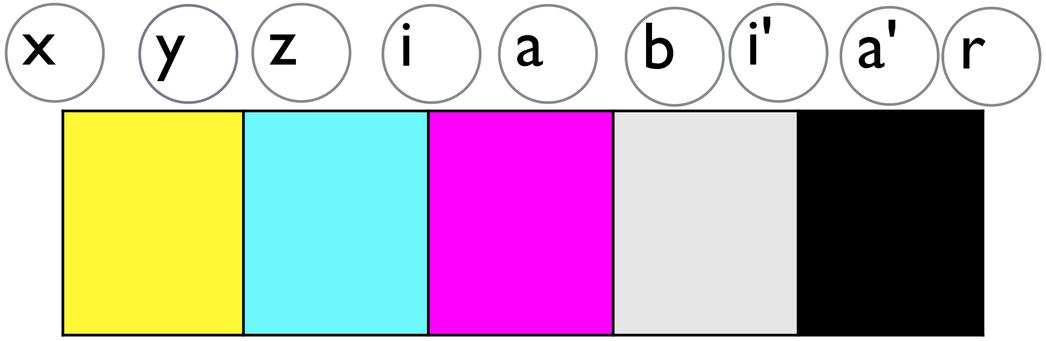


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

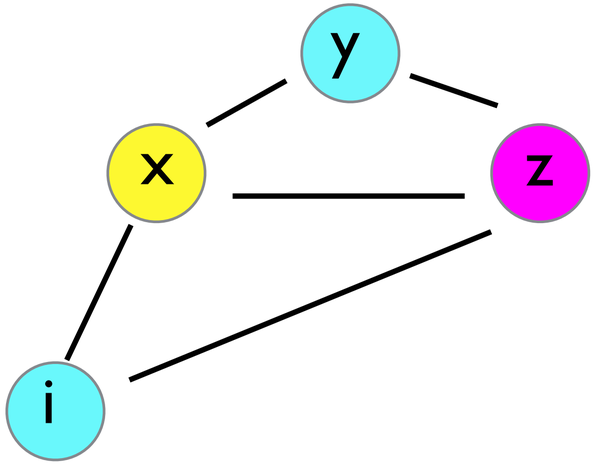


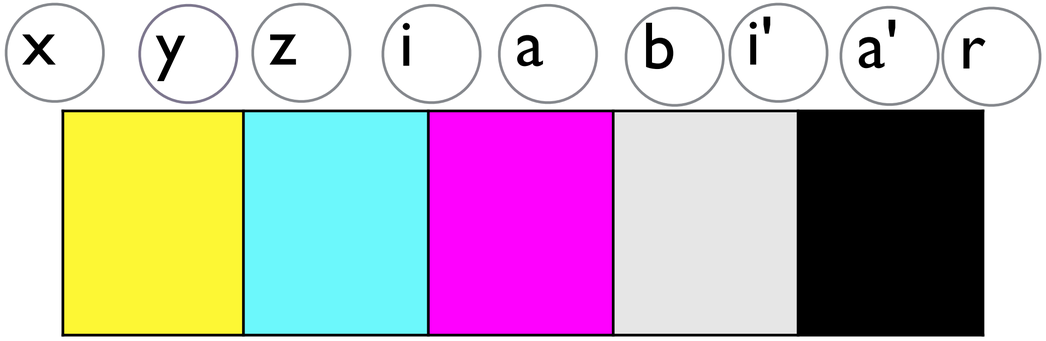


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

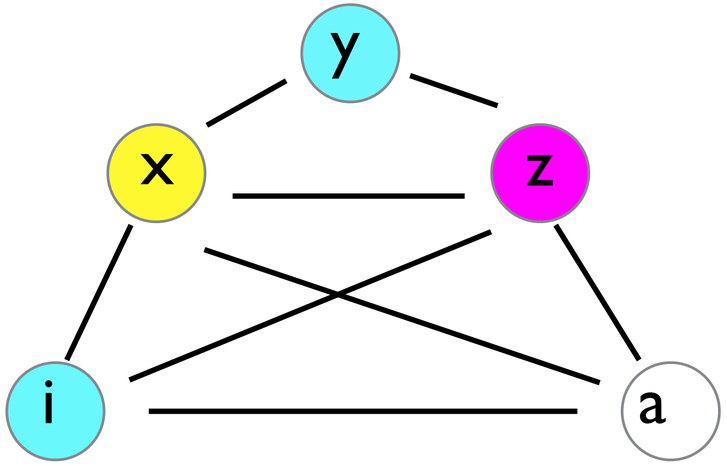


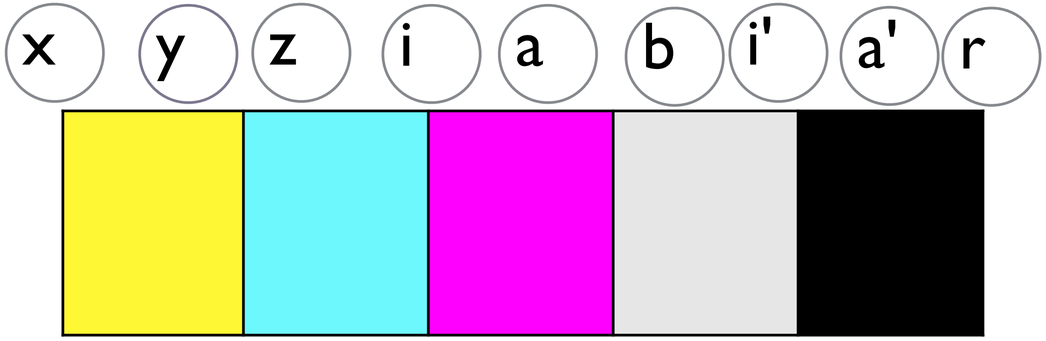


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

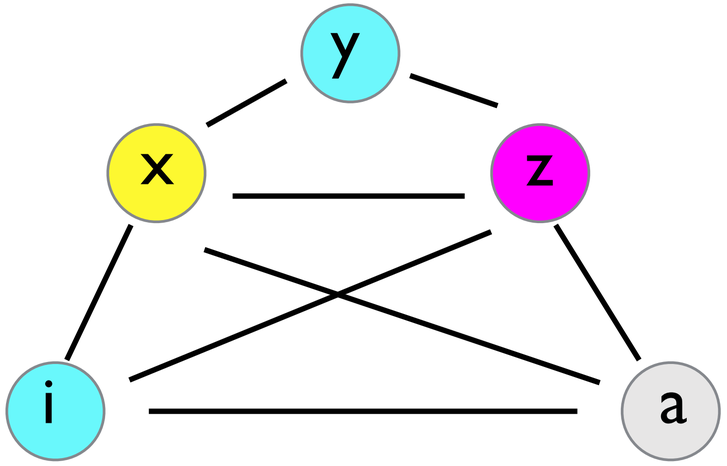


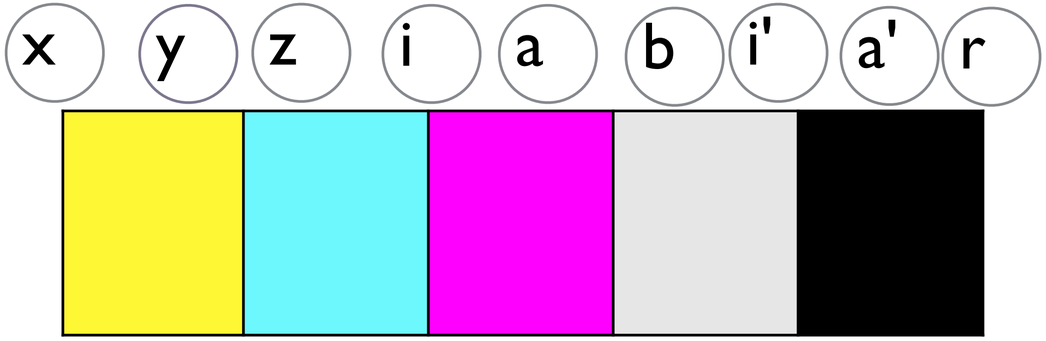


SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph

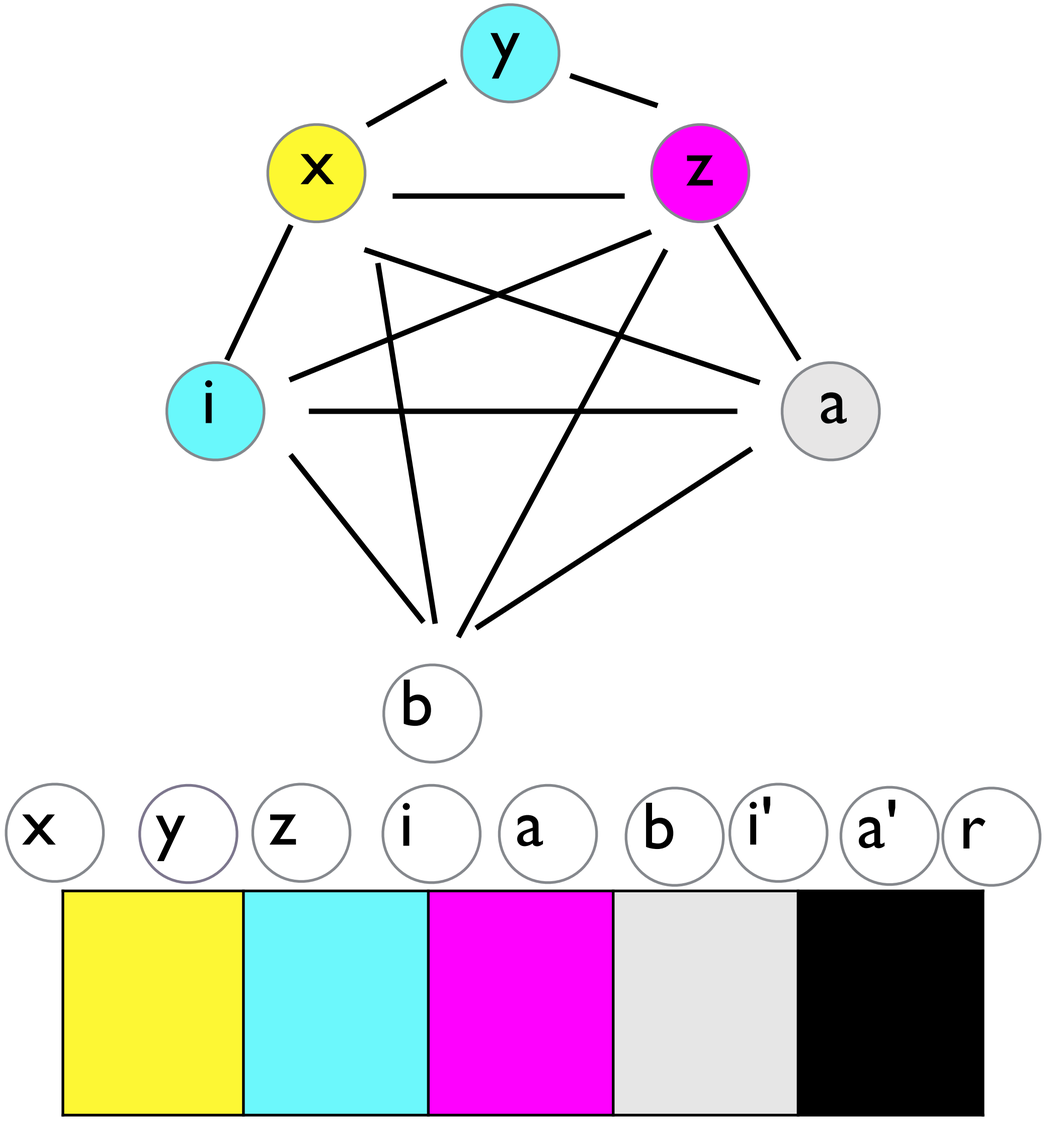




SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

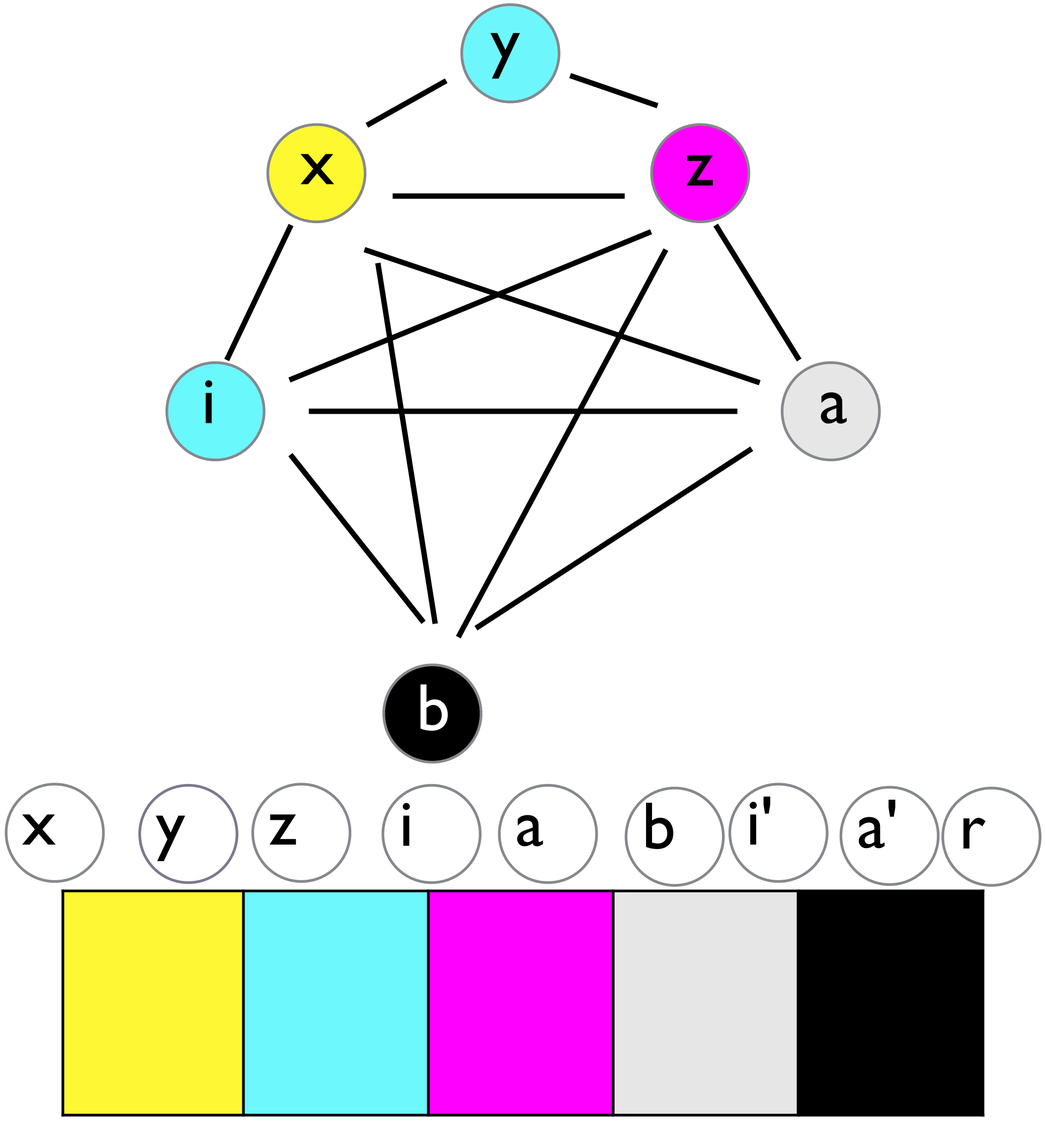
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

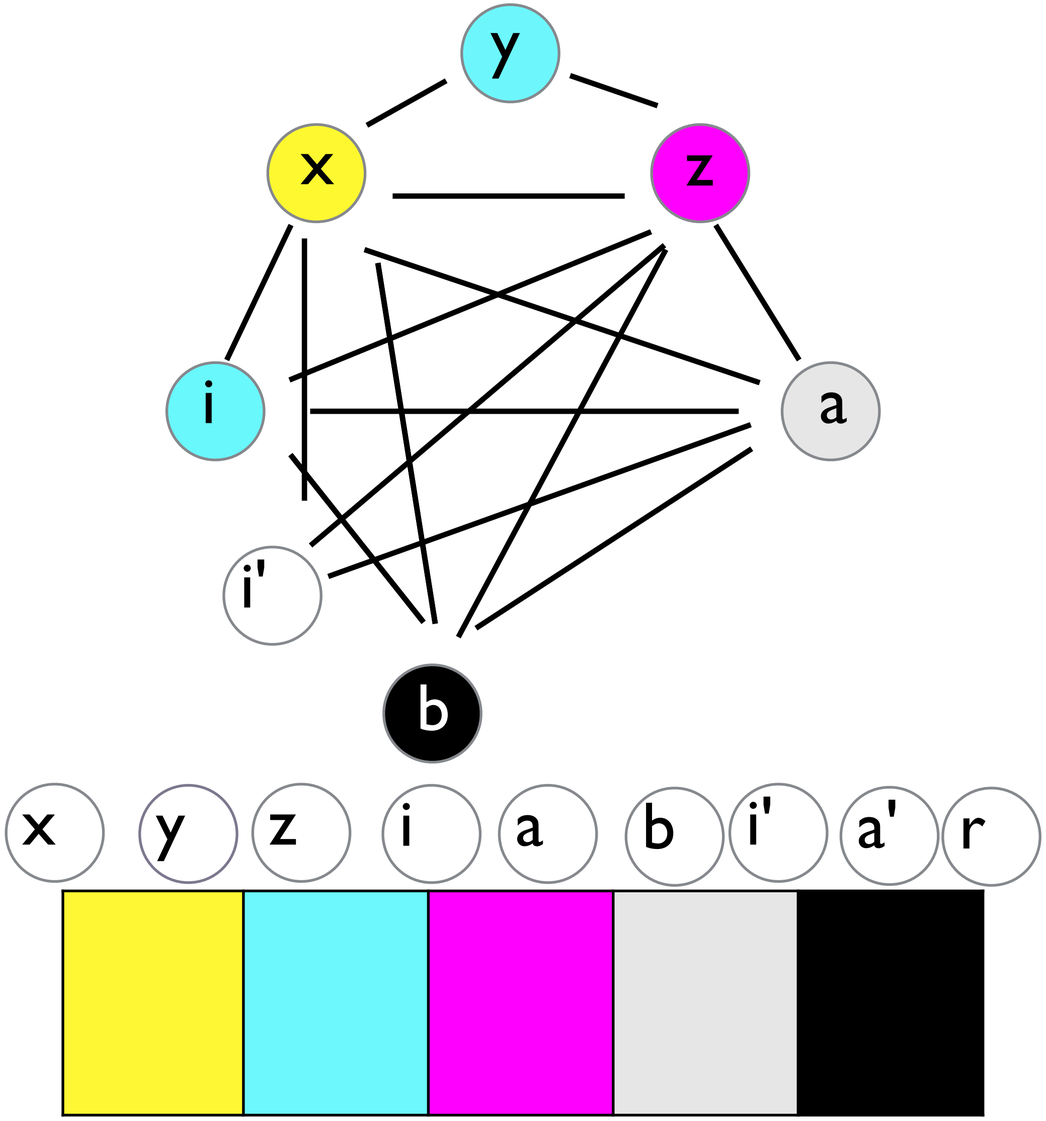
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

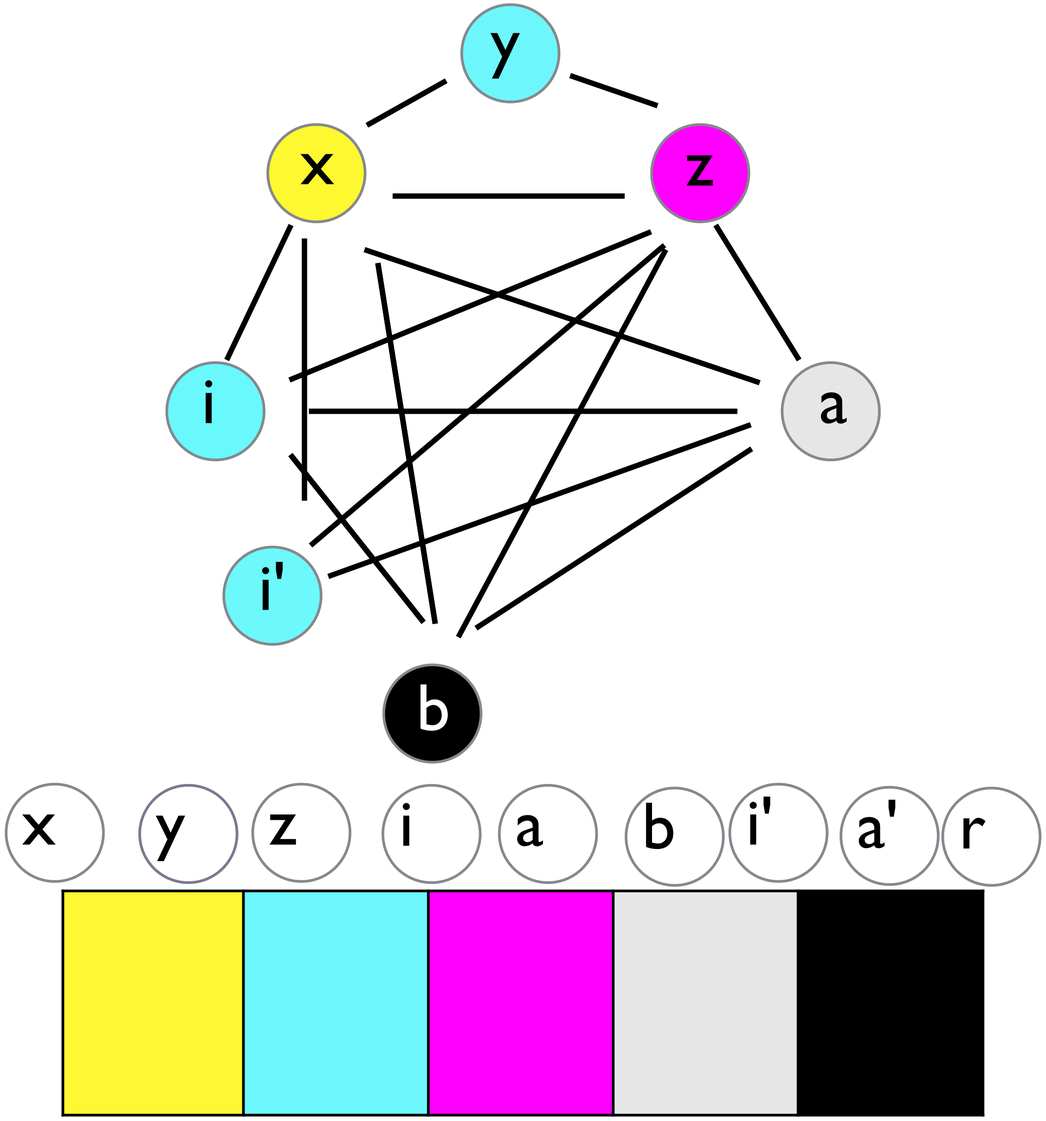
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

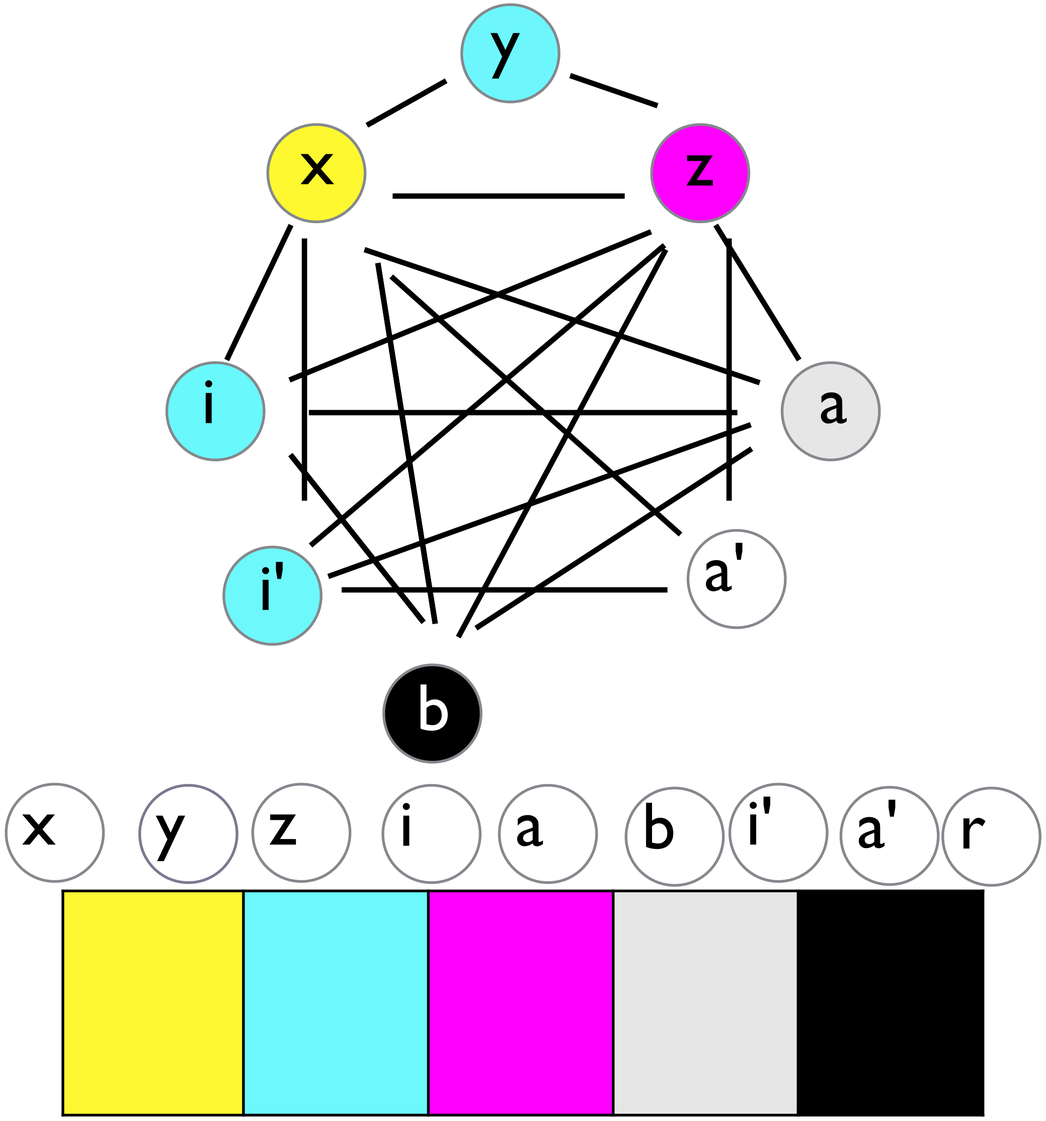
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

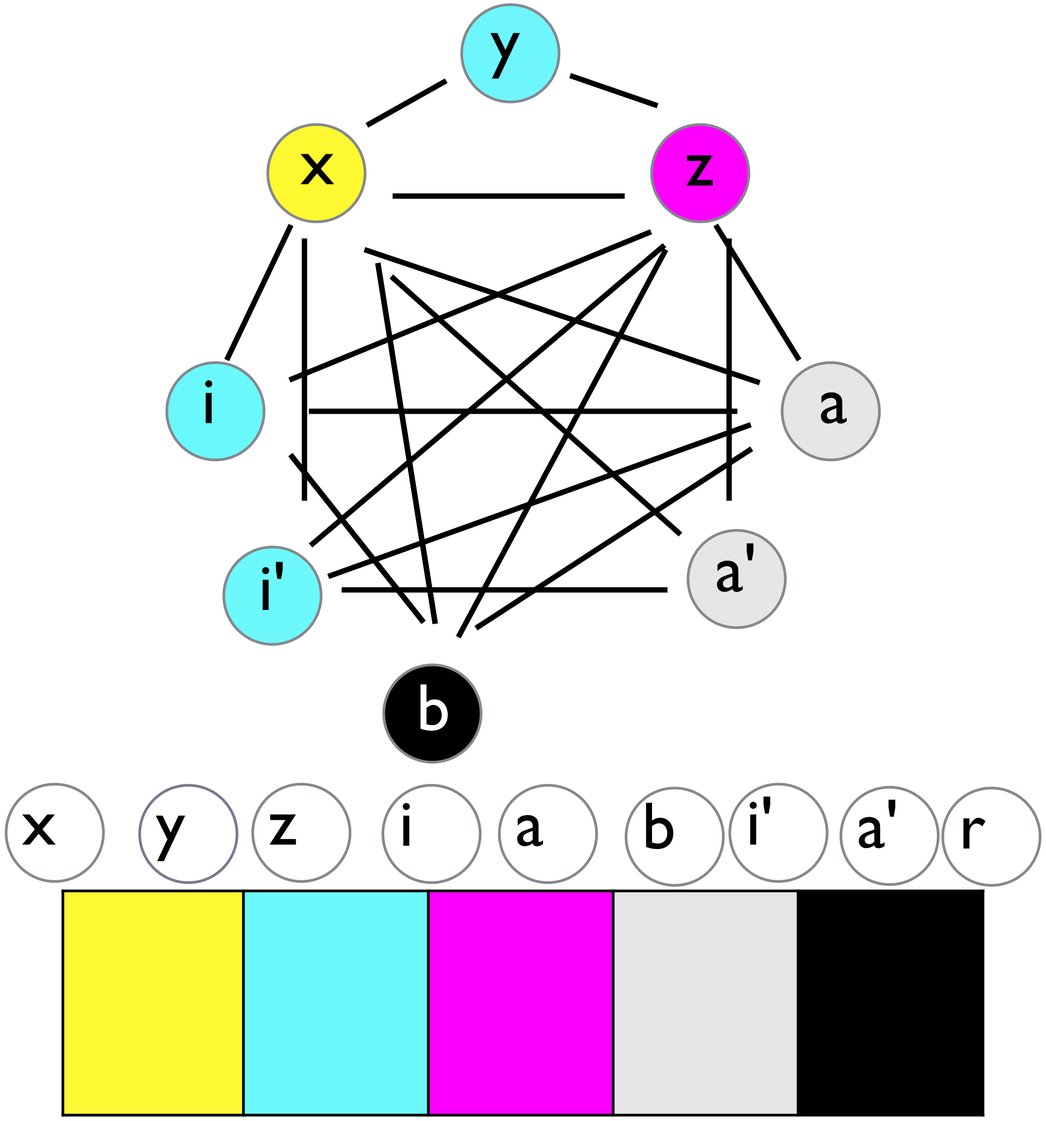
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

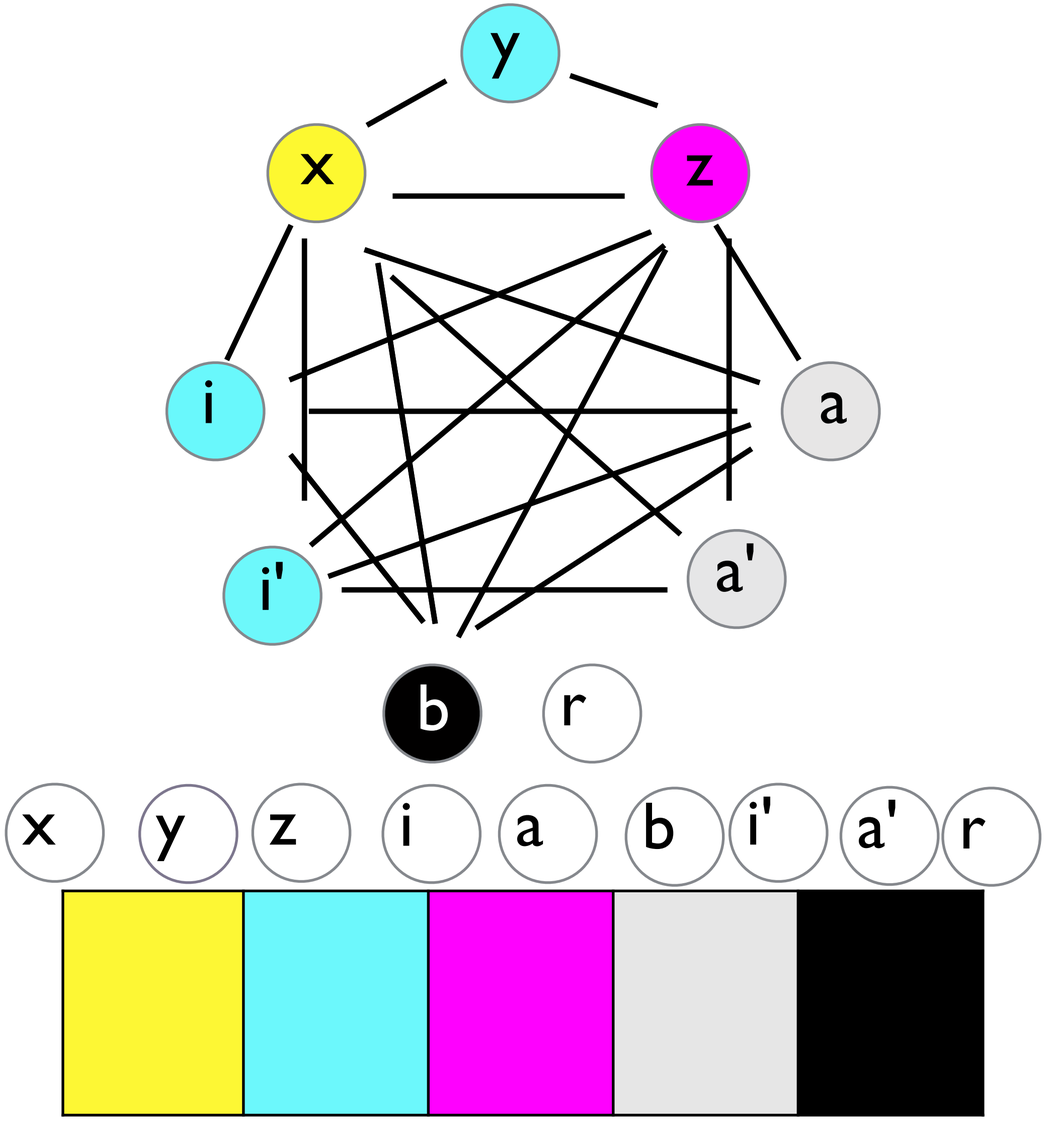
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

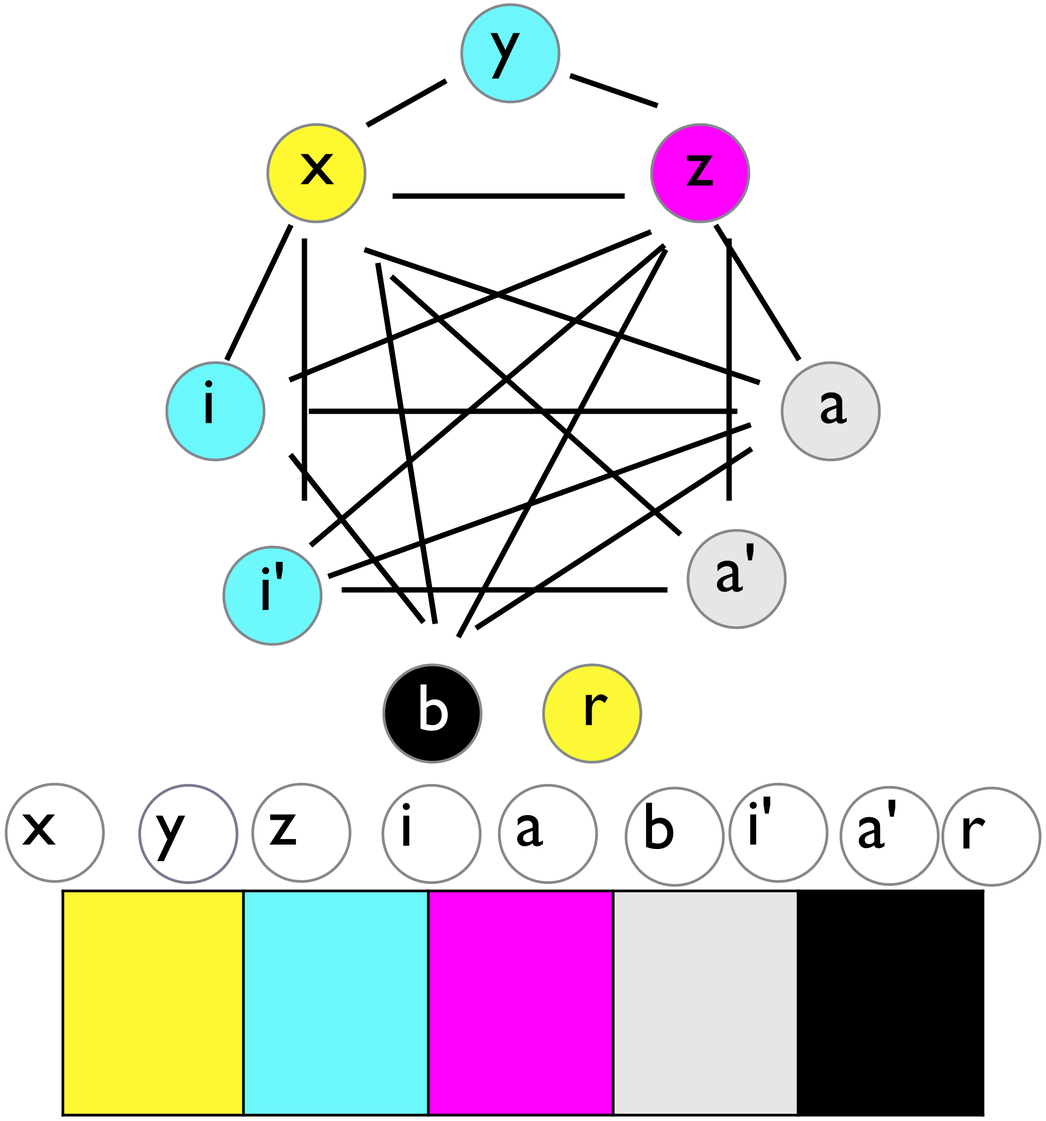
Interference Graph



SSA Program

```
f(x,y,z):
  loop(i,a):
    thn():
      r = a * z
      ret r
    els():
      i' = i - 1
      a' = a + x
      br loop(i', a')
    b = i == 0
    cbr b thn() els()
  br loop(y, 0)
```

Interference Graph



Incorporating Register Allocation

Perform register allocation on all the **blocks** of our SSA program. Treat function blocks specially.

Effects on Codegen

Store results directly in the output register

x = y - z

mov rx, ry

sub rx, rz

Does this always work? Need to be careful if output register is the same as one of the input registers (e.g., rx = rz)

Can either use a scratch register, or op-specific tricks:

sub rx, ry

imul rx, -1

Spilling

If a variable is picked to be spilled:

- When it is assigned to, store the result in memory
- When it is used, access memory

One issue:

x = y + z

If all x,y,z are spilled, cannot implement this without a register.

Easy solution: reserve one scratch register for this purpose:

```
mov r, [rsp - off(y)]
add r, [rsp - off(z)]
mov [rsp - off(x)], r
```

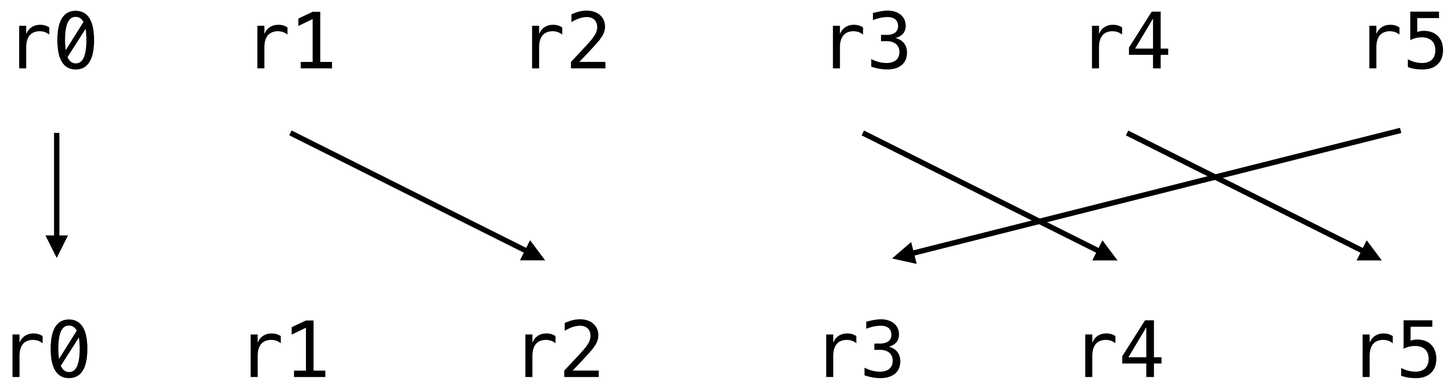
Implementing Branch with Arguments

```
f(a,b,c): ...
...
br f(x,y,z)
```

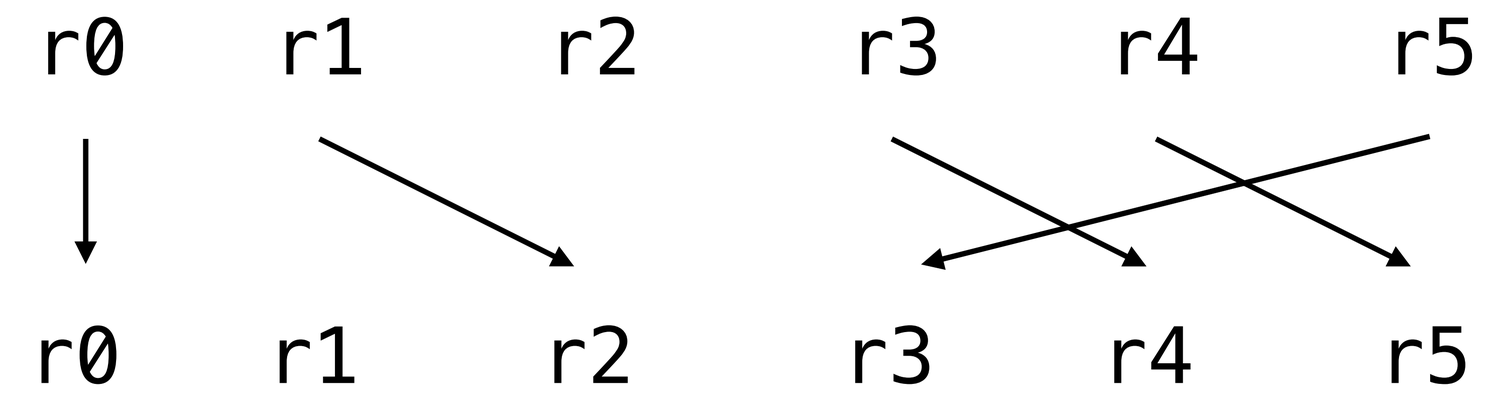
```
mov r_x, r_a
mov r_y, r_b
mov r_z, r_c
jmp f
```

what if a,b,c registers and
x,y,z registers overlap?

Implementing Branch with Arguments

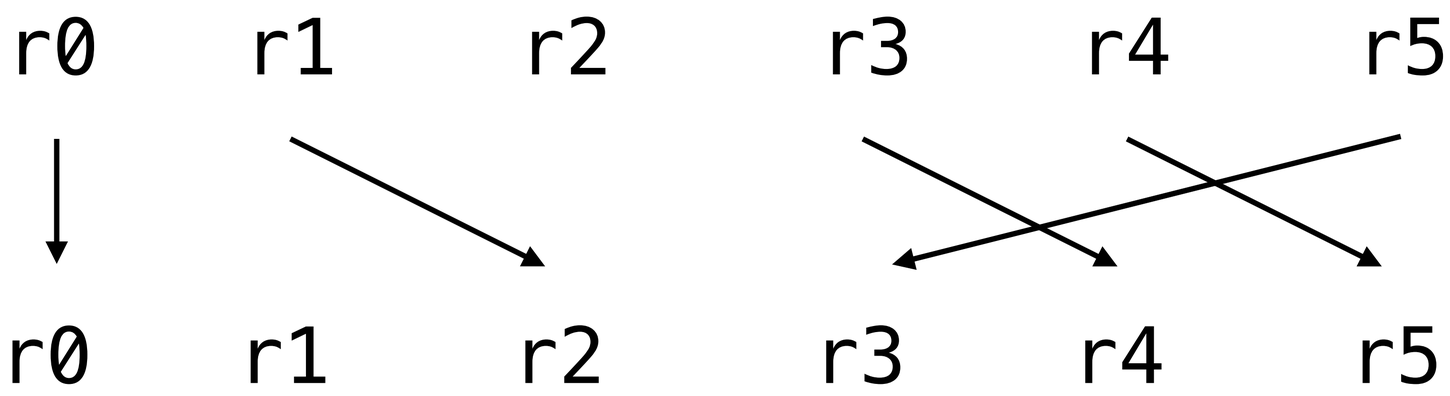


Implementing Branch with Arguments



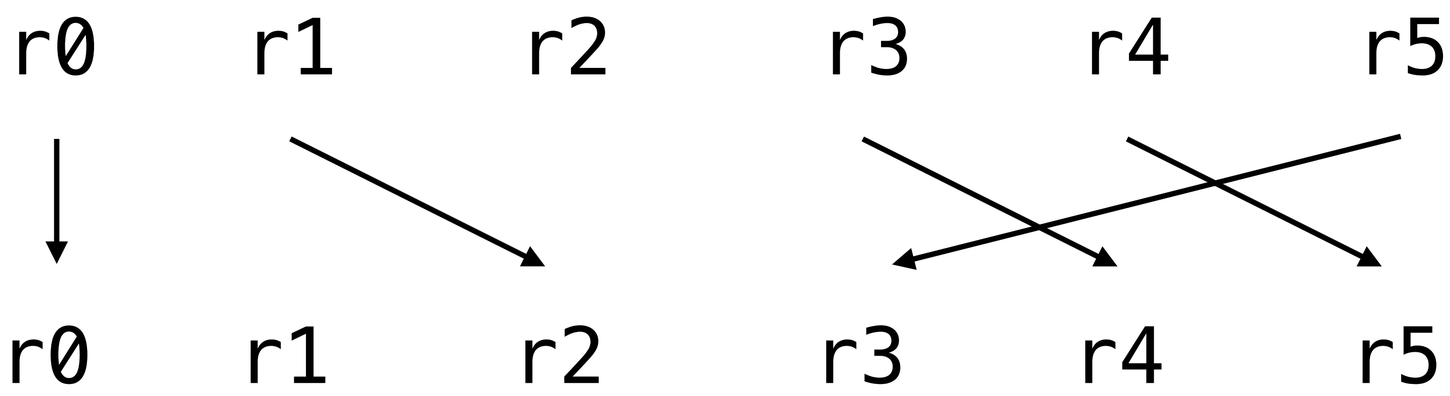

```
nop
```

Implementing Branch with Arguments



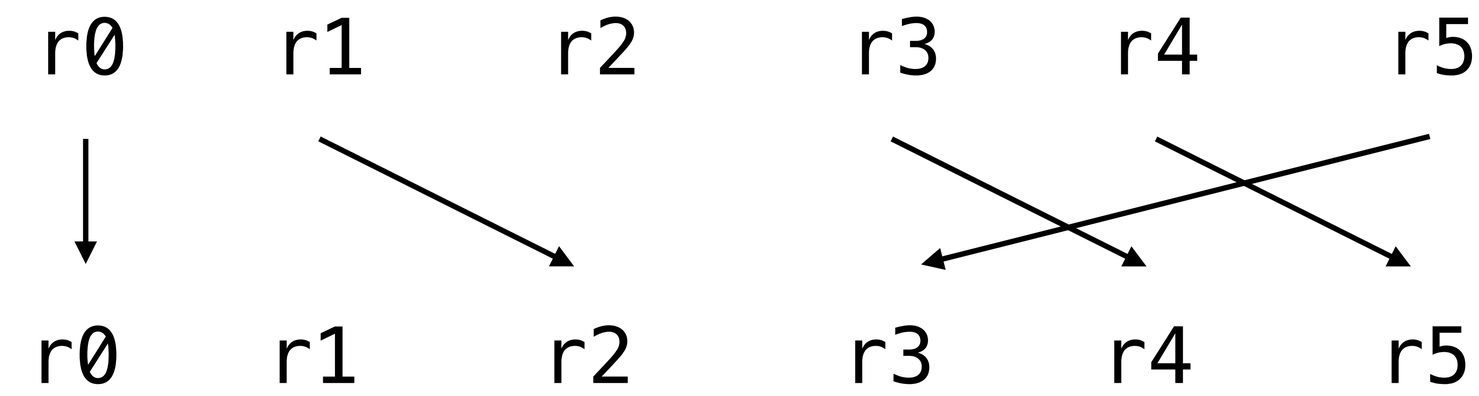

```
nop  mov r2, r1
```

Implementing Branch with Arguments




```
nop  mov r2, r1
```

Implementing Branch with Arguments




```
nop   mov r2, r1
```

```
xchg r3, r4
xchg r3, r5
```

xchg is like mov, but **exchanges** the values without need for an extra register.
Faster than xor swapping

SSA reg allocation is polytime, but minimizing the resulting number of movs/xchg is NP hard

Register Allocation vs Calling Conventions

Now that we are using registers we need to take care to respect treatment of registers in the calling conventions we use.

Register Allocation vs Calling Conventions

Now that we are using registers we need to take care to respect treatment of registers in the calling conventions we use.

In System V AMD 64 Calling convention, registers are divided into two classes:

- **volatile** aka **caller-save**: when you make a call, the value of these registers may change when the callee returns
- **non-volatile** aka **callee-save**: when you make a call, the value of these registers will be the same when the callee returns

Volatile/Caller Save registers

volatile aka **caller-save**

x = ...

y = f(z)

z = x + y

if **x** is stored in a volatile register, its value may be overwritten by the function **f**.

- Simple solution: save all live volatiles to the stack before a call, restore after the call
- Better solution: add nodes to interference graph for volatile registers, add conflicts at every non-tail call

Non-volatile/Callee Save registers

```
y = ...
z = x + y
ret z
```

if **y** is stored in a **non-volatile** register, the value of the register must be **saved** on entry and **restored** when we return

- Solution: save all used non-volatiles to the stack at the beginning of every global function def, restore them before every return/external tail call
- Start spilled variables **after** the saved non-volatile registers

Implementing function blocks

```
fun f_fun(a,b,c,...):
  br f_tail(a,b,c,...)
```

1. Save all used non-volatiles/callee-save registers
2. Treat f_fun's args are pre-determined by the calling convention, otherwise similar to any branch with args

```
mov [rsp - 8], rbx
mov [rsp - 16], rbp
...
; br f_tail(...)
```

Implementing ret

```
ret x
```

- Move x into rax
- Restore non-volatile/callee-saves

mov rax, loc(x)

mov rbx, [rsp - 8]

mov rbp, [rsp - 16]

...